URMACK · FLAGSHIP PRODUCT · CONFIDENTIAL

Urmack Watch

What to read before writing code, the security risks that can sink this product, the standards we hold ourselves to, and the controls we build with — from day one, not after the first incident.

Document 05 — Pre-Build Reading & Security Playbook
Companion documents 01 System Architecture · 02 Flows & SDLC Roadmap · 03 Product Designs · 04 Tech Stack & Chain Strategy
Version 1.0 · 16 October 2026
Author Office of the Interim CTO, Urmack
Status Approved for build — gates referenced from Doc 01 §2 and Doc 04 §9

1. How to use this document

Documents 01–04 say what we are building and with what. This one answers three questions a builder must settle *before* week 1: **what do I read first, what can go wrong, and which security standards and measures actually apply to us?** §2 is the reading list in priority order; §3 the risk register; §4 the standards we adopt vs. defer; §5 the concrete controls in the codebase; §6 the compliance timeline; §7 disclosure duties. External sources were verified in October 2026 and are listed in §8.

2. Reading list — in priority order

2.1 Internal (read all four before writing code)

DOCUMENT	READ FOR	SECTION THAT MATTERS MOST
01 — System Architecture	The seven bounded contexts, control vs. evaluation plane, data model, severity model	§7 failure modes & mitigations; §10 explicit non-goals
02 — Flows & SDLC Roadmap	Phase gates, the dry-run gate, spike pass/fail discipline	Flow 1 (evaluation pipeline), Flow 2 (onboarding)
03 — Product Designs	What the client portal must make self-evident to a non-security operator	Invariant editor + evidence viewer screens
04 — Tech Stack & Chain Strategy	Chain order, finality states, cost model, 90-day build order	§2.1 finality policy; §9 build sequence

2.2 External — before the component that consumes it

SOURCE	WHY WE READ IT	WHEN
Ethereum.org — Proof of Stake (slot/epoch/finality)	The correctness of our Ethereum finality policy depends on getting this exactly right	Week 1 — chain adapter
docs.base.org, docs.optimism.io, docs.arbitrum.io — finality & sequencer pages	Safe/finalized semantics per L2; batch cadence assumptions we alert on	Week 1–2 — L2 adapter
L2BEAT — Layer 2 summary	Sequencer centralization and stage ratings we must not misrepresent in marketing	Quarterly — chain risk review
viem docs · BullMQ docs · Foundry Book (Anvil, cheatcodes)	The three libraries the evaluation plane is built on	Week 1 — spikes

SOURCE	WHY WE READ IT	WHEN
Supabase — Row Level Security guidance (grants *and* policies)	RLS is our tenant boundary; the documented failure modes are the ones that leak data	Week 7 — tenancy
NIST SSDF SP 800-218 (v1.1)	Our CI/CD and vulnerability-handling practices, expressed as adoptable practices rather than a compliance project	Week 1 — repo/CI bootstrap
OWASP ASVS 5.0	The authorization, tenant-separation and API test checklist the portal/API must satisfy	Before API work — week 7
OWASP Top 10 (2025) · OWASP Smart Contract Top 10 (2025)	Developer awareness; the latter shapes which invariant categories our library must cover	Week 5 — invariant library
OpenZeppelin Contracts 5.x security docs · ethereum.org smart-contract security guide	Reference patterns (access control, proxies, reentrancy) behind the contracts clients monitor	Week 5 — spec authoring
Immunefi Severity Classification v2.3 · Code4rena submission guidelines	The evidence-pack format we borrow: location, impact, reproduction, stated severity criteria	Week 5–6 — evidence pack
FIRST CVSS v4.0 spec · MITRE CWE · CISA KEV	Severity for vulnerabilities in *our own* software and dependency triage — not for invariant breaches	Ongoing — vuln handling
OWASP Dependency-Track · OSV.dev · CISA SBOM hub · SLSA v1.2 build requirements	Supply-chain controls: SBOM per release, continuous dependency analysis, build provenance	Before first release
GDPR Art. 28/33 (consolidated) · EIOPA DORA overview	Alert routes contain personal data; EU fund buyers bring ICT third-party due diligence	Before Beta (GDPR) · Phase 3 (DORA contracts)

> **Deliberately not on the list:** SWC Registry (unmaintained since 2020 — legacy cross-reference only), tokenomics papers, and any vendor whitepaper claiming complete exploit detection. Secureum is useful training material, not a normative standard.

3. Security risk register

Ranked by damage × likelihood for this specific product. Every risk names the gate that closes it; risks without a gate are risks we have decided to live with knowingly — and we do not have any of those.

ID	RISK	MITIGATION (BUILT-IN, NOT BOLTED ON)	CLOSED AT
R1	**False assurance**	The worst failure mode: a protocol is "green" while insolvent. Mitigations: honest non-goals in all docs (Doc 01 §10), heartbeat dead-man on our own blindness, finality states exposed on every alert, detection-latency SLO published rather than implied.	Phase 1 gate
R2	**Monitoring blind spot (RPC outage, reorg, provider split-view)**	Two independent RPC vendors from day one with circuit breakers; degrade cadence instead of skipping; multi-provider head disagreement raises a data-quality alert; reorg triggers re-evaluation and loud retraction.	Phase 1 gate
R3	**Tenant data leak via RLS gap**	`org_id` on every row; request-path DB role is never `service_role`; cross-tenant read/write probes are CI tests; pooled-connection transaction context reviewed; RLS pen-test is a Beta exit gate.	Phase 2 gate
R4	Spec author error → false-positive storm	Dry-run backfill over 7 days of history before arming; plans immutable and versioned; dedup + cooldown + per-route rate caps; anomaly channel never pages.	Onboarding flow (Doc 02 Flow 2)
R5	**Supply-chain compromise (dependency, CI)**	pnpm lockfile + `pnpm audit` in CI, Dependabot/Renovate with review gates, no unreviewed postinstall scripts, SBOM (CycloneDX) per release, SLSA v1.2 Build L1 provenance → L2, OIDC-only CI credentials.	Before first release

ID	RISK	MITIGATION (BUILT-IN, NOT BOLTED ON)	CLOSED AT
R6	**Secret exposure (RPC keys, DB creds, webhook HMAC keys)**	GitHub OIDC → cloud secret manager, zero long-lived CI keys; `age/SOPS` -encrypted config for local dev; provider keys scoped per chain and rotated quarterly; HMAC keys per route; secrets never logged (structured-log redaction enforced).	Week 1 bootstrap
R7	**Insider compromise of Urmack**	**No signing keys exist anywhere in the system** — the maximum blast radius is false alerts, not stolen funds; read-only RPC credentials; append-only audit log on spec/route/severity changes with step-up re-auth; quarterly access review.	Continuous
R8	Alert-channel abuse: spoofed or phishing alerts to customers	HMAC-SHA256 signature + ±5 min timestamp window on every webhook; published verify snippets; alerts never carry auth links or credentials; fixed message shape so operators can recognize genuine Urmack alerts.	Week 5–6
R9	Evidence-pack tampering or loss	Immutable object storage, 90-day retention, 15-minute signed URLs, content hash recorded with the alert; export bundle for IR retainers.	Week 5–6
R10	Our own availability (a silent monitor is worse than none)	Evaluation SLO 99.9% of expected ticks measured in Grafana Cloud; dead-man alert on heartbeat; 24-hour shadow run gate on any evaluation-plane deploy.	Phase 1 gate
R11	Personal data in alert routes (GDPR)	Names, emails, channel identifiers and IPs are minimized and retention-bounded; processor agreement + subprocessor list before Beta; breach notification to controller without undue delay.	Phase 2 gate
R12	Reputation risk from overstated capability	Marketing language reviewed against Doc 01 §10: we sell economic-invariant monitoring, not complete exploit detection. L2 sequencer claims match L2BEAT stage, not aspiration.	Continuous

4. Standards we focus on — adopt now vs. defer

For a 5–10 person team, standards are only useful where they change what we build. This is the cut.

STANDARD / FRAMEWORK	DECISION	WHAT WE TAKE FROM IT	TRIGGER / TIMING
NIST SSDF SP 800-218	ADOPT NOW	Security ownership, protected build access, change review, release testing, vulnerability intake/response — practices folded into our existing CI, not a side project	Week 1 repo bootstrap
OWASP ASVS 5.0	ADOPT NOW	Requirement checklist for authn/authz, tenant separation, API + webhook handling; selected Level 1/2 controls traced to CI tests	Week 7 API + tenancy
OWASP Top 10 (2025) + Smart Contract Top 10 (2025)	ADOPT NOW	Developer awareness; smart-contract edition drives which invariant categories our library must ship	Ongoing / invariant library
SLSA v1.2	ADOPT NOW	Build L1 (provenance tied to release), targeting L2 (authenticated provenance, hosted build isolation)	Before first release
SBOM (CISA minimum elements) + **Dependency-Track/OSV** scanning	ADOPT NOW	CycloneDX SBOM per release, continuous advisory scanning, patch/exception SLAs, KEV prioritized over raw CVSS	Before first release
CVSS v4.0 + CWE	ADOPT NOW	Score *and* vector for vulnerabilities in our own software; never used as the severity model for invariant breaches (see §4.1)	Ongoing
OWASP SAMM 2.2	LIGHTWEIGHT	Short baseline self-assessment, a few improvements per quarter; no formal maturity scoring yet	Quarterly
GDPR (processor role)	ADOPT NOW	Processor terms, retention limits on alert payloads/logs, breach notice to controller without undue delay	Before Beta
SOC 2 Type II	DEFER → PHASE 3	Not legally triggered by size — it is buyer-triggered. 6–12 months incl. observation window; ~$12–20k audit fees plus readiness. Start readiness only when a design partner's procurement demands it	Trigger: enterprise deal

STANDARD / FRAMEWORK	DECISION	WHAT WE TAKE FROM IT	TRIGGER / TIMING
ISO/IEC 27001:2022	DEFER	Keep a lightweight risk-based ISMS (this document is its seed); accredited certification only if EU/international buyers value it over SOC 2	Trigger: buyer demand
EU DORA	MONITOR	In force since Jan 2025; we are not automatically in scope, but as an ICT provider to EU financial entities expect due-diligence, incident-cooperation and exit clauses in contracts — prepare the paperwork, not a compliance program	Phase 3 EU deals
NIS2	MONITOR	Scope depends on our own service category and national implementation; do not assume "SaaS vendor" qualifies. Re-check before EU expansion	Advisory

4.1 Severity model alignment

Two vocabularies, never conflated: **CVSS v4.0 + CWE** for vulnerabilities in our own software and dependencies, and our own **Critical / Warning / Anomaly** economic severity (Doc 01 §6) for invariant breaches — a breach may deserve a Critical page with no CVE in existence. Following Immunefi/Code4rena reporting discipline, every alert carries: affected chain and contract, invariant with expected vs. observed values, block/transaction references, reproduction inputs, confidence and stated limitations, and a private response route. **Observed breach, potential exploit, and monitoring/data-quality failure are three distinct labels** and never share an alert without saying which is which.

5. Security measures we build with

Platform & tenancy

- Postgres RLS on every table; `org_id` scoping; no `service_role` in request context
- Cross-tenant isolation tests in CI; RLS pen-test as a Phase 2 gate
- Supabase Auth (magic link → passkeys → SSO); HTTP-only session cookies; step-up re-auth for routing/severity changes
- Per-org API rate limits; per-route alert rate caps
- `Cache-Control: no-store` on all tenant-bearing responses

Keys & secrets

- **No signing material anywhere** — read-only RPC credentials only, scoped per chain
- GitHub OIDC → cloud secret manager; no long-lived CI keys
- age/SOPS encrypted local config; quarterly rotation; per-route webhook HMAC keys

Supply chain

- Lockfile pinning, `pnpm audit`, Dependabot with review gates
- CycloneDX SBOM per release + VEX notes for non-affected advisories
- SLSA v1.2 L1 provenance (→ L2), OIDC-issued build identities
- KEV-first prioritization for dependency patches; patch SLA by severity

Application

- ASVS-traced tests: authn, authz, tenant separation, input validation on spec YAML
- Zod validation at every trust boundary; typed ABIs — no runtime `eval` of client specs
- Spec compiler sandboxes assertion evaluation; plans immutable + checksummed
- Webhook signing with timestamp window; published verify snippets (JS/Python/Go)
- Structured logging with secret redaction; no payload bodies in logs

Operations

- OpenTelemetry + evaluation SLO (99.9% of expected ticks); dead-man heartbeat
- 24-hour shadow-run gate for evaluation-plane deploys
- PITR Postgres backups, cross-region evidence replication, quarterly restore drill
- Append-only audit log; quarterly access review
- Incident runbook with severity-1 path and customer notification templates

Data & disclosure

- Evidence packs immutable, 90-day retention, 15-min signed URLs, content hash on alert
- Processor terms + subprocessor list before Beta; retention limits on route payloads
- Public security contact (`security@`) with a published disclosure policy (90-day coordinated)
- Never publish exploitable detail before the protocol has patched

6. Compliance timeline (what happens when, and what triggers it)

PHASE	SECURITY DELIVERABLES	GATE
Phase 0 — Spike	Secrets bootstrap (OIDC, SOPS), SSDF ownership assigned, CI with lint/typecheck/tests + dependency audit	Repo green
Phase 1 — Alpha	ASVS-mapped API tests, RPC-key scoping, RLS policies + cross-tenant CI probes, IR runbook v1, honest-capability docs review	30-day correct-evaluation run; zero cross-tenant findings
Phase 2 — Beta	RLS pen-test, SBOM per release, SLSA L1 provenance, GDPR processor terms, disclosure policy published, restore drill	Pen-test passed (Doc 01 §2)
Phase 3 — GA	SOC 2 readiness assessment → Type II fieldwork if buyer-triggered; DORA contract clauses for EU financial entities; SLSA L2	Enterprise deal closure

7. Disclosure & incident response duties

- **A vulnerability in Urmack** (e.g. an RLS bypass): coordinated disclosure page, 90-day clock, customer notification with evidence of containment; if customer personal data is affected, notify controllers without undue delay.
- **A breach detected in a customer's protocol** (our product working): severity-classified alert + evidence pack, IR playbook attach, and — per our disclosure principles — no public detail until the protocol acts.
- **Our own monitoring failure** (blind spot): heartbeat dead-man alert, customer notification, post-incident write-up. Reporting our own failures is the trust behavior competitors selling fear cannot copy.

8. Sources

Standards & compliance: csrc.nist.gov/projects/ssdf (SP 800-218 v1.1) · owasp.org/projects/asvs (5.0) · owasp.org/projects/top-ten (2025) · owasp.org/projects/smart-contract-top-10 · owasp.org/projects/samm (2.2) · slsa.dev/spec/v1.2/ · cisa.gov/topics/.../sbom (2026 minimum-elements updates) · owasp.org/projects/dependency-track · osv.dev · cisa.gov/known-exploited-vulnerabilities-catalog · aicpa-cima.com/resources/landing/system-and-organization-controls-soc-suite-of-services · drata.com/learn/soc-2/cost (planning ranges, not market prices) · iso.org/standard/27001
Severity & disclosure: first.org/cvss/specification-document (CVSS v4.0) · cwe.mitre.org · immunefi.com/immunefi-vulnerability-severity-classification-system-v2-3/ · docs.code4rena.com/competitions/submission-guidelines · swcregistry.io (legacy since 2020)
Regulation: eur-lex.europa.eu CELEX 32022R2554 (DORA) · eiopa.europa.eu DORA overview · eur-lex.europa.eu CELEX 32022L2555 (NIS2) · digital-strategy.ec.europa.eu NIS2 update (Jan 2026 proposal — verify national enactment) · eur-lex.europa.eu CELEX 02016R0679 (GDPR, Art. 28/33)
Chain & contract security: ethereum.org/developers/docs/smart-contracts/security/ · docs.openzeppelin.com/contracts/5.x/ · l2beat.com/layer2s/summary · ethereum.org/developers/docs/consensus-mechanisms/pos/